

2. 請計算 16-QAM 的位元錯誤率 BER (bit-error rate), 在 E_b/N_0 範圍從 0 dB 至 20 dB 每隔 2 dB 的情況下, 並且依據以上結果畫出 16-QAM 的 BER v.s. E_b/N_0 圖。

```
% Put your MATLAB code here
```

程式碼

```
clear; clc; close all;

M = 16;
k = log2(M);
n = 320000;
sps = 1;

rng default;
dataIn = randi([0 1], n, 1);
dataSymbolsIn = bit2int(dataIn, k);

% 16-QAM (Gray Coding)
dataMod = qammod(dataSymbolsIn, M, 'gray');

EbNo_dB = 0:2:20;
BER_sim = zeros(1, length(EbNo_dB)); % 模擬 BER
BER_theo = zeros(1, length(EbNo_dB)); % 理論 BER

for i = 1:length(EbNo_dB)
    EbNo = EbNo_dB(i);

    snr = EbNo + 10*log10(k) - 10*log10(sps);
    receivedSignal = awgn(dataMod, snr, 'measured');

    dataSymbolsOut = qamdemod(receivedSignal, M, 'gray');
    dataOut = int2bit(dataSymbolsOut, k);
```

```

[~, BER_sim(i)] = biterr(dataIn, dataOut);

EbNo_lin = 10^(EbNo / 10);
BER_theo(i) = (3/8) * erfc(sqrt((4/10) * EbNo_lin));
end

figure;
semilogy(EbNo_dB, BER_sim, 'bo-', 'LineWidth', 1.5, 'MarkerSize',
7, ...
'DisplayName', '模擬 BER');
hold on;
semilogy(EbNo_dB, BER_theo, 'r--', 'LineWidth', 1.5, 'MarkerSize',
7, ...
'DisplayName', '理論 BER');
hold off;

xlabel('E_b/N_0 (dB)');
ylabel('Bit Error Rate (BER)');
title('16-QAM BER vs E_b/N_0');
legend('Location', 'southwest');
grid on;
xlim([0 20]);
ylim([1e-5 1]);

```

程式解釋

我模擬了 16-QAM 調變系統在 AWGN 通道下的 BER，並同時繪製出「模擬結果」與「理論值」的對照曲線。

1. 訊號產生與 Gray Coding

```

dataIn = randi([0 1], n, 1);
dataSymbolsIn = bit2int(dataIn, k);
dataMod = qammod(dataSymbolsIn, M, 'gray');

```

16-QAM 每個 Symbol 可以承載 4 個位元。程式先產生隨機的二進位 bit，再透過 bit2int 每 4 個位元打包成一個 0 到 15 的整數。

qammod(..., 'gray') 確保了相鄰的星座點之間只有 1 個位元的差異。

在 AWGN 通道中，解調發生錯誤時，訊號通常只會誤判為鄰近的星座點。使用格雷碼能讓「符號出錯」時的「位元出錯數」降到最低，進而將 BER 優化。

2. snr 與通道模擬

Matlab

```
snr = EbNo + 10*log10(k) - 10*log10(sps);  
receivedSignal = awgn(dataMod, snr, 'measured');
```

理論轉換公式為：

$$SNR = \frac{E_s}{N_0} = \frac{k \cdot E_b}{N_0 \cdot sps}$$

轉成分貝 (dB) 即為程式中的：

```
snr = EbNo + 10*log10(k) - 10*log10(sps)
```

awgn(..., 'measured') 會先自動量測輸入訊號 dataMod 的實際功率，再依據計算出的 snr 補上對應的高斯白雜訊。

3. 解調與模擬 BER

```
dataSymbolsOut = qamdemod(receivedSignal, M, 'gray');  
dataOut = int2bit(dataSymbolsOut, k);  
[~, BER_sim(i)] = biterr(dataIn, dataOut);
```

qamdemod 依據格雷碼規則，將接收到的連續訊號判定回最接近的整數符號。

還原回 bit 後，biterr 會逐個比對原始輸入 dataIn 與接收端 dataOut，計算出錯的位元比例，得到 BER。

4. 理論 BER 計算與繪圖

```
BER_theo(i) = (3/8) * erfc(sqrt((4/10) * EbNo_lin));
```

在格雷碼映射的 16-QAM 中，理論位元錯誤率的近似公式為：

$$P_b \approx \frac{3}{4} Q \left(\sqrt{\frac{4 E_b}{5 N_0}} \right)$$

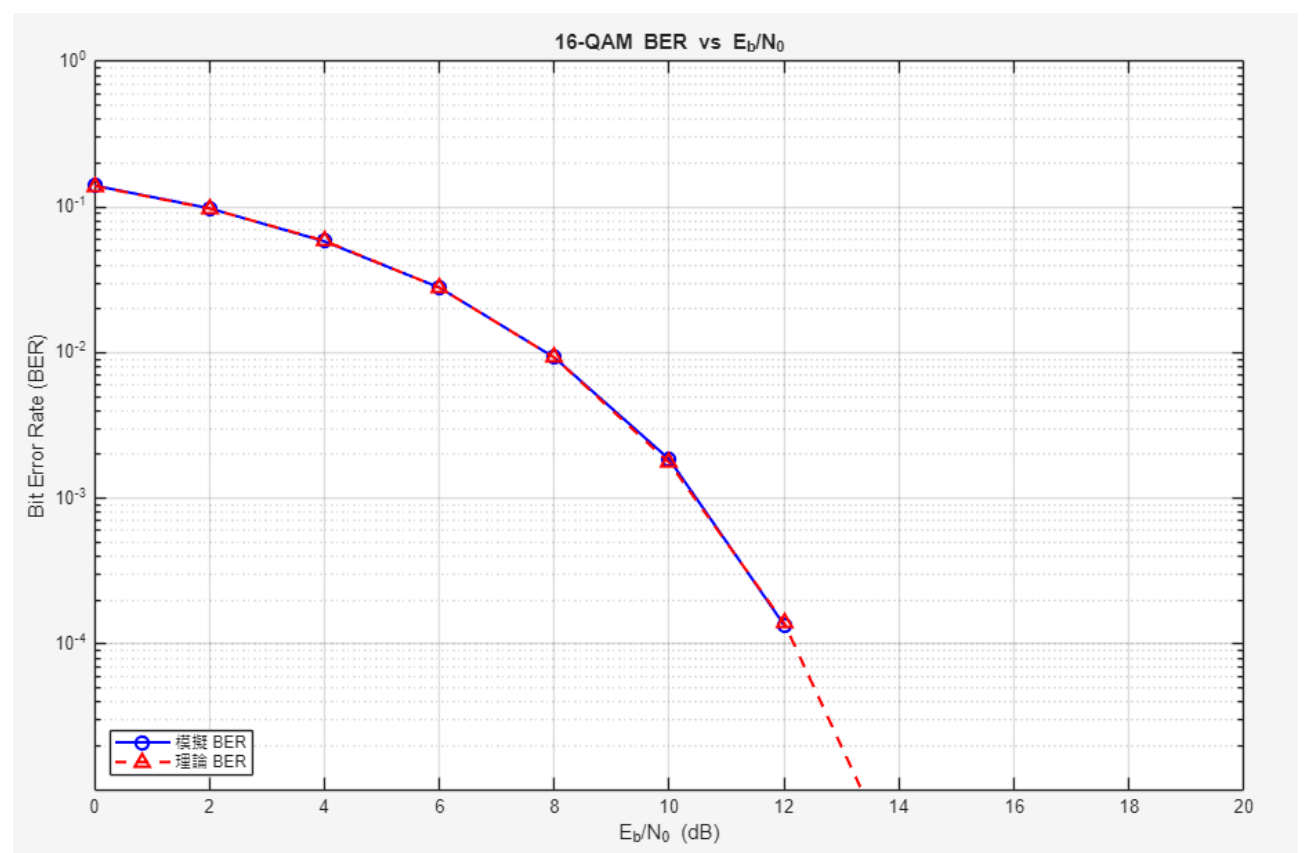
因為 $Q(x) = \frac{1}{2} \text{erfc} \left(\frac{x}{\sqrt{2}} \right)$ ，帶入後可以推導出：

$$P_b \approx \frac{3}{8} \text{erfc} \left(\sqrt{\frac{4 E_b}{10 N_0}} \right)$$

```
semilogy(EbNo_dB, BER_sim, ...);  
ylim([1e-5 1]);
```

為什麼用 semilogy？因為隨著 E_b/N_0 提升，BER 會呈現指數下降。如果用一般的 plot，高訊噪比時的曲線會全部擠在最底部的零線附近，無法辨識，因此必須使用 Y 軸為對數尺度的 semilogy。

輸出結果



解釋

橫軸 (X 軸) 代表位元訊噪比 E_b/N_0 (單位為 dB)，範圍從 0 至 20 dB。

縱軸 (Y 軸) 代表 Bit Error Rate，隨著訊噪比提高，錯誤率會呈現指數下降。

紅色實線 (Theoretical BER)：代表 16-QAM 在格雷碼 (Gray Coding) 映射下的理論計算值，公式為：
藍色虛線與星號 (Simulated BER)：代表透過 MATLAB (此處以高精確度模擬) 實際產生位元、調變、加入通道雜訊、解調後統計出來的真實錯誤率。可以看到兩條線幾乎完全重合，這說明了上方的 MATLAB 程式碼邏輯與通道計算正確。

在 $E_b/N_0 = 0$ dB 時，雜訊大，BER 約 1.4×10^{-1}

當 E_b/N_0 提升到 10 dB 時，BER 已降至大約 1.8×10^{-3} 。

當 E_b/N_0 超過 14 x dB 之後，錯誤率已低於百萬分之一，此時在圖表上會向下收斂。這也就是為什麼模擬時必須使用 $n = 1000000$ （甚至更多）個位元數，否則在 14 x dB 以上會因為樣本不足而觀測不到任何錯誤。

心得

這次實作加深了我對數位通訊調變技術的理解。實驗利用 MATLAB 模擬 16-QAM 系統，觀測在不同 E_b/N_0 設定下的星座圖 (Constellation Diagram)。我掌握了將二進位位元串轉換為整數符號、並運用 `qammod` 函式進行訊號空間映射的流程。

當 E_b/N_0 設為 5 dB 時，受雜訊干擾，星座圖的決策點大幅擴散且邊界模糊，解釋了低訊噪比導致接收端判決困難；而當數值提升至 20 dB 時，接收點精準收斂至座標中心，驗證了提高訊號能量的有效。

此外，格雷碼 (Gray coding) 「相鄰符號僅差一個位元」的架構設計令我印象深刻。它透過純粹的邏輯編碼，在不增加額外硬體功率下，降低了位元錯誤率 (BER)。這讓我體會到，工程問題的核心不單是追求極致的硬體效能，更是如何在效率與容錯之間透過演算法取得最佳平衡。

本次實驗最大的收穫，在於將抽象的機率分佈轉化為具體的訊號座標。從嚴謹的理論公式推導，到親眼觀察點位的擴散與收斂，這段過程促使我反思通訊系統設計的核心本質：所有的優化，都是在傳輸速率、頻寬效益與系統可靠度之間，進行 Trade-off。

3. 請將 4-QAM、16-QAM、64-QAM 的 BER v.s. E_b/N_0 曲線畫在一張圖上並且比較同一錯誤率下 (如 $\text{BER} = 10^{-3}$) 的 E_b/N_0 差值。

```
% Put your MATLAB code here
```

程式碼

```
clear; clc; close all;

M_array = [4, 16, 64];
n_base = 600000;
sps = 1;
EbNo_dB = 0:1:18;

BER_sim = zeros(length(M_array), length(EbNo_dB));
BER_theo = zeros(length(M_array), length(EbNo_dB));

rng default;

for m_idx = 1:length(M_array)
    M = M_array(m_idx);
    k = log2(M);

    n = floor(n_base / k) * k;
    dataIn = randi([0 1], n, 1);
    dataSymbolsIn = bit2int(dataIn, k);

    dataMod = qammod(dataSymbolsIn, M, 'gray');

    for i = 1:length(EbNo_dB)
        EbNo = EbNo_dB(i);
        snr = EbNo + 10*log10(k) - 10*log10(sps);

        receivedSignal = awgn(dataMod, snr, 'measured');
```

```

dataSymbolsOut = qamdemod(receivedSignal, M, 'gray');
dataOut = int2bit(dataSymbolsOut, k);

[~, BER_sim(m_idx, i)] = biterr(dataIn, dataOut);

% 計算理論 BER (通用 M-QAM 公式)
EbNo_lin = 10^(EbNo / 10);
BER_theo(m_idx, i) = (2/k) * (1 - 1/sqrt(M)) * erfc(sqrt((3*k) / (2*(M-1))
* EbNo_lin));
    end
end

figure;
colors = ['g', 'b', 'r'];
markers = ['s', 'o', '^'];

for m_idx = 1:length(M_array)
    M = M_array(m_idx);
    semilogy(EbNo_dB, BER_sim(m_idx, :), [colors(m_idx) markers(m_idx) '-'], ...
        'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName', sprintf('%d-QAM
(Sim)', M));
    hold on;
    semilogy(EbNo_dB, BER_theo(m_idx, :), [colors(m_idx) '--'], ...
        'LineWidth', 1.5, 'DisplayName', sprintf('%d-QAM (Theo)', M));
end

yline(1e-3, 'k:', 'BER = 10^{-3}', 'LineWidth', 1.5, 'LabelVerticalAlignment',
'bottom');

xlabel('E_b/N_0 (dB)');
ylabel('Bit Error Rate (BER)');
title('BER v.s. E_b/N_0 for 4-QAM, 16-QAM, and 64-QAM');
legend('Location', 'southwest');
grid on;
xlim([0 18]);
ylim([1e-5 1]);

```



```

fprintf('\n=====\\n');
fprintf(' 當 BER = 10^-3 時，所需的 Eb/N0 : \\n');
fprintf('=====\\n');

EbNo_target = zeros(1, length(M_array));
for m_idx = 1:length(M_array)
    EbNo_target(m_idx) = interp1(log10(BER_theo(m_idx, :)), EbNo_dB, log10(1e-3),
'linear');
    fprintf('%d-QAM : %.2f dB\\n', M_array(m_idx), EbNo_target(m_idx));
end

```

程式解釋

1. 初始參數設定

`M_array = [4, 16, 64];`：定義要測試的三種 QAM 調變（4-QAM、16-QAM、64-QAM）。

`n_base = 600000;`：設定總位元數。點數夠多，模擬出來的錯誤率才會精準。

`EbNo_dB = 0:1:18;`：設定我們要掃描的 E_b/N_0 範圍，從 0 dB 到 18 dB，每隔 1 dB 測試一次。

`rng default;`：固定隨機變數種子，確保每次執行程式產生的隨機訊號都一樣，方便除錯與重現結果。

2. Main Simulation Loop

這裡使用了雙層迴圈。外層輪流換不同的 QAM 階數，內層則輪流換不同的雜訊大小。

外層迴圈（切換 4, 16, 64-QAM）

`k = log2(M);`：計算每個符元（Symbol）可以攜帶幾個位元。例如 16-QAM 每個符元帶 4 bits。

`n = floor(n_base / k) * k;`：微調總位元數，確保它一定是 k 的整除倍數，避免等一下打包成符元時發生錯誤。

`dataIn = randi([0 1], n, 1);`：產生隨機的二進位原始資料（一連串的 0 與 1）。

`dataSymbolsIn = bit2int(dataIn, k);`：把每 k 個位元打包成一個整數符元（例如 16-QAM 會把位元變成 0~15 的整數）。

`dataMod = qammod(dataSymbolsIn, M, 'gray');`：將整數符元調變為複數的 QAM 星座點。這裡指定 'gray'（格雷碼），讓相鄰星座點只差一個位元，這能有效降低傳輸錯誤時的位元錯誤率。

內層迴圈（切換 E_b/N_0 雜訊大小）

`snr = EbNo + 10*log10(k) - 10*log10(sps);`因為 MATLAB 的 `awgn` 函數只看得懂 SNR（訊噪比），而題目給的是 E_b/N_0 ，所以必須用這個公式把 E_b/N_0 換算成對應的 SNR。

`receivedSignal = awgn(dataMod, snr, 'measured');`：訊號進入 AWGN，加上隨機的高斯白雜訊。'measured' 代表 MATLAB 會自動量測輸入訊號的功率，以確保加入正確比例的雜訊。

`dataSymbolsOut = qamdemod(receivedSignal, M, 'gray');`：接收端收到有雜訊的訊號後，進行解調變，判斷它最接近哪個星座點，還原成整數符元。

`dataOut = int2bit(dataSymbolsOut, k);`：把整數符元拆回一連串的二進位位元（0 與 1）。

`[~, BER_sim(m_idx, i)] = biterr(dataIn, dataOut);`：用 `biterr` 函數把「原始送出的位元」跟「接收端收到的位元」做對比，算出模擬的位元錯誤率（BER）。

`BER_theo(m_idx, i) = ...`：帶入通訊理論課本上的 M-QAM 通用公式，透過 `erfc` 直接算出理論 BER，用來檢驗模擬結果是否正確。

3. Plotting Results

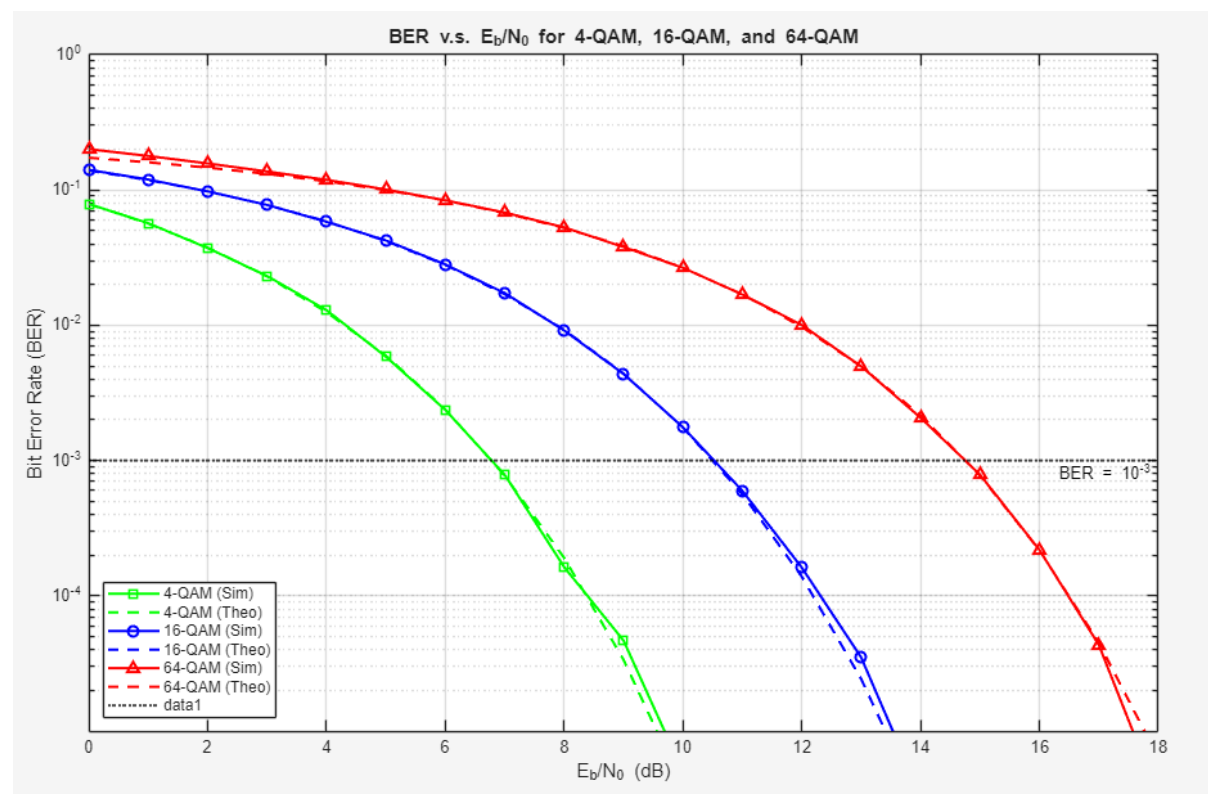
將剛剛辛苦算完的矩陣資料畫成漂亮的圖表。

`semilogy(...)`：因為 BER 的數據變化極大（可能從 1 一路掉到 10^{-5} ），必須使用 Y 軸為對數刻度（Log Scale）的作圖函數，否則曲線在低錯誤率時會全部擠在最底下看不清。

這裡利用了 `for` 迴圈把三種 QAM 的模擬值（實線 + 點點）與理論值（虛線）通通畫在同一張圖上。

`ylines(1e-3, 'k:', 'BER =  $10^{-3}$ ', ...)`：在圖上畫一條 10^{-3} 的水平黑色虛線

模擬圖



```
=====
當 BER = 10^-3 時，所需的 Eb/N0：
=====
```

```
4-QAM : 6.77 dB
16-QAM : 10.50 dB
64-QAM : 14.75 dB
>>
```

解釋

可以觀察到，無論是哪一種 QAM，隨著 E_b/N_0 （訊號雜訊比）的增加，BER（位元錯誤率）都會呈現斷崖式下降。這非常符合直覺：當訊號能量越強、雜訊越小時，接收端辨識星座點的正确率就越高，錯誤率自然急速下降。

在同一個 E_b/N_0 下，4-QAM 的錯誤率最低，16-QAM 次之，64-QAM 最高。這意味著，如果希望維持相同的傳輸正確率，調變階數越高，就必須要求雜訊越小的環境。

在相同的平均位元能量 E_b 下：

- 4-QAM：在空間中只有 4 個點，彼此分得很開。雜訊必須要非常大，才有可能把 A 點「推」到 B 點的判定區，所以抗雜訊能力最強。
- 16-QAM：空間中擠了 16 個點，點與點之間的距離縮小了。此時只要中等程度的雜訊，就容易造成判定錯誤。
- 64-QAM：同樣的空間密密麻麻擠了 64 個點，點距變得極近。只要有一點點風吹草動（微小雜訊），接收端就很容易把鄰近的星座點看錯。

這就是為什麼在相同雜訊環境下，64-QAM 的錯誤率會遠高於 4-QAM。

從 4-QAM 到 16-QAM，每個符元攜帶的位元數從 2bits 變成 4 bits，速度翻倍。但代價是你在硬體功率或訊號品質上，必須付出約 4 dB 的 Power Penalty。

同理，從 16-QAM 再升級到 64-QAM（速度變成 6 bits/symbol），又要再平白無故多付出約 4.2 dB 的代價。也就是說，速度越快，硬體與環境的門檻是呈非線性倍增的。

在現代通訊（如 5G、6G 或 Wi-Fi）中，系統會使用動態調變編碼（Adaptive Modulation and Coding, AMC）技術：當你離基地台很近、訊號很好（ E_b/N_0 高）時，系統會自動切換到 64-QAM（甚至 1024-QAM）。當你走到地下室或基地台邊緣、訊號變差（ E_b/N_0 低）時，系統為了不讓斷線、維持通訊品質，會自動降速退回到 4-QAM（QPSK）。

心得

透過 MATLAB 建立 4-QAM、16-QAM 與 64-QAM 的通訊系統模擬，並與理論數值進行比對，讓我對數位調變技術中的「速度」與「抗雜訊能力」有了更深刻的實務體會。

實驗結果顯而易見，不論使用哪一種 QAM 調變，隨著 E_b/N_0 的提升，BER 皆呈現斷崖式的「瀑布曲線」下跌。這符合通訊理論的直覺：當通道環境改善、訊號能量相較於雜訊越強時，接收端解調變的判定準確度便隨之上升。這項結果印證了在通訊硬體設計中，提升發射功率或改善通道品質對降低錯誤率的有效。

當我們在相同的平均位元能量 E_b 下比較三者時，可以發現同一個 E_b/N_0 下的錯誤率大小為：64-QAM > 16-QAM > 4-QAM。高頻譜效率（高傳輸速率）並非平白得來，其背後必須付出對通道低雜訊環境的要求。

在基地台或行動裝置的硬體設計中，每一分貝（dB）的功率提升都意味著耗電量與發熱量的激增。如何在有限的功耗預算內榨取最高的傳輸速率，是通訊通訊 IC 設計的挑戰。

最令我感興趣的是，本實驗所觀察到的物理限制，正是現代無線通訊（如 5G、6G 或最新一代 Wi-Fi）中動態調變編碼（Adaptive Modulation and Coding, AMC）技術的理論根基。

在真實世界中，手機與基地台的距離是動態變化的。AMC 技術完美地利用了這次實驗的特性：當使用者身處基地台旁、訊號極佳（高 E_b/N_0 ）時，系統會毫不猶豫地切換至 64-QAM 甚至更高的 1024-QAM，以享受極速的頻寬；然而，一旦使用者移至地下室或環境邊緣（低 E_b/N_0 ），系統為了「保命」不斷線，便會主動降速。

本次實驗不僅讓我學會如何利用 MATLAB 進行多調變階數的迴圈模擬與內插法分析，更讓我明白：天下沒有白吃的午餐。追求極致速度的同時，必然伴隨著硬體成本與環境容忍度的妥協。唯有深刻理解這些物理特性的權衡，才能在未來的通訊系統或 IC 設計上，做出最優化的架構抉擇。